2) Write a C program that contains a string (char pointer) with a value "Helloworld". The program should AND or XOR each character in this string with 127 and display the result.

===== 1) XOR =====

```
#include <stdio.h>
#include <string.h>

int main()
{
   char String[] = "Helloworld";
   char cipher[100];
   char plain[100];
   int len = strlen(String);

   for (int i = 0; i < len; i++)
   {
      cipher[i] = String[i] ^ 127;
   }
   cipher[len] = '\0';
   printf("Cipher text: %s\n", cipher);

   for (int i = 0; i < len; i++)
   {
      plain[i] = cipher[i] ^ 127;
   }
   plain[len] = '\0';
   printf("Plain text: %s\n", plain);

   return 0;
}
```

Output:-
Cipher text: 7<non-printable characters>
Plain text: Helloworld

===== 2) AND =====

```
#include <stdio.h>
#include <string.h>

int main()
{
   char String[] = "Helloworld";
   char cipher[100];
```

```
    char plain[100];
    int len = strlen(String);

    for (int i = 0; i < len; i++)
    {
        cipher[i] = String[i] & 127;
    }
    cipher[len] = '\0';
    printf("Cipher text: %s\n", cipher);

    for (int i = 0; i < len; i++)
    {
        plain[i] = cipher[i] & 127;
    }
    plain[len] = '\0';
    printf("Plain text: %s\n", plain);

    return 0;
}
```

Output:-

Cipher text: Helloworld
Plain text: Helloworld

8) Write a C/JAVA program to implement the Rijndael algorithm logic.

```
import java.security.*;
import javax.crypto.*;
import javax.crypto.spec.*;
import java.io.*;

public class AES {
   public static String asHex(byte buf[]) {
      StringBuffer strbuf = new StringBuffer(buf.length * 2);
      int i;
      for (i = 0; i < buf.length; i++) {
         if (((int) buf[i] & 0xff) < 0x10)
            strbuf.append("0");
         strbuf.append(Long.toString((int) buf[i] & 0xff, 16));
      }
      return strbuf.toString();
   }

   public static void main(String[] args) throws Exception {
      String message = "AES still rocks!!";

      KeyGenerator kgen = KeyGenerator.getInstance("AES");
      kgen.init(128);

      SecretKey skey = kgen.generateKey();
      byte[] raw = skey.getEncoded();
      SecretKeySpec skeySpec = new SecretKeySpec(raw, "AES");

      Cipher cipher = Cipher.getInstance("AES");
      cipher.init(Cipher.ENCRYPT_MODE, skeySpec);
      byte[] encrypted = cipher.doFinal((args.length == 0 ? message : args[0]).getBytes());
      System.out.println("encrypted string: " + asHex(encrypted));

      cipher.init(Cipher.DECRYPT_MODE, skeySpec);
      byte[] original = cipher.doFinal(encrypted);
      String originalString = new String(original);
      System.out.println("Original string: " + originalString + " " + asHex(original));
   }
}
```

Output:-

encrypted string:
16158125f4e148dd668d23bfa768ff7948b574373a0c0ea1a34e79a1492d61af
Original string: AES still rocks!! 414553207374696c6c20726f636b732121

13) Write a Java program to Implement RSA algorithm.

```
import java.util.*;
import java.math.*;

class RSA {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the prime number p: ");
        BigInteger p = sc.nextBigInteger();
        System.out.print("Enter the prime number q: ");
        BigInteger q = sc.nextBigInteger();

        BigInteger n = p.multiply(q);
        System.out.println("The value of n is: " + n);

        BigInteger Pn = p.subtract(BigInteger.ONE).multiply(q.subtract(BigInteger.ONE));
        System.out.println("The value of Pn is: " + Pn);

        BigInteger a, b;
        System.out.println("e values ...");
        for (int i = 1; i < Pn.intValue(); i++) {
            a = BigInteger.valueOf(i);
            b = Pn;
            if (a.gcd(b).intValue() == 1)
                System.out.print(" " + a);
        }
        System.out.println();

        System.out.print("Select any e value: ");
        BigInteger e = sc.nextBigInteger();
        BigInteger d = e.modInverse(Pn);
        System.out.println("The value of d is: " + d);
        System.out.println("Public key Pu{e,n}: " + "{" + e + "," + n + "}");
        System.out.println("Private key Pr{d,n}: " + "{" + d + "," + n + "}");

        System.out.print("Enter plain text M: ");
        BigInteger M = sc.nextBigInteger();
        BigInteger C = M.modPow(e, n);
        System.out.println("Your Plain Text M is: " + M);
        System.out.println("Your Cipher Text C is: " + C);

        BigInteger M1 = C.modPow(d, n);
        System.out.println("Your decrypted plain text M is: " + M1);

        sc.close();
    }
```

}

Output:-

```
Enter the prime number p: 5
Enter the prime number q: 11
The value of n is: 55
The value of Pn is: 40
e values ...
 1 3 7 9 11 13 17 19 21 23 27 29 31 33 37 39
Select any e value: 3
The value of d is: 27
Public key Pu{e,n}: {3,55}
Private key Pr{d,n}: {27,55}
Enter plain text M: 5
Your Plain Text M is: 5
Your Cipher Text C is: 15
Your decrypted plain text M is: 5
```